

dAIgnose: An AI-Based Mobile Application for Image-Based Health Diagnosis

Emir Ekinci

Istanbul, Turkiye

eknciemir@outlook.com

Abstract—One of the most beneficial services that society can receive is the healthcare system. However, access to healthcare services is decreasing today due to population density and ignorant visits. In addition, overwhelming work environment decreasing quality and efficiency of care provided.

dAIgnose offers patients AI-based visual diagnosis, aims to reduce unnecessary hospital visits and provides better work environment for healthcare professionals. Additionally, it enables beneficial features such as appointment scheduling and lab report tracking.

Skin, nail, and oral diseases can be remotely diagnosed with dAIgnose, which utilizes the TensorFlow Lite framework. Users can upload photos of their symptoms to receive preliminary diagnoses, obtain information about the severity of their conditions, and receive guidance regarding potential doctor visits.

Public Kaggle datasets used to train diagnose models with Python. Flutter is used for user interface and as a feature of TensorFlow Lite, diagnose model runs on device which allows users to diagnose without internet connection. Google Firebase services are used to handle authentication and database operations.

On-device experimental results show that the model achieved 71% accuracy on validation data and 20% on real-world examples, producing results in 10 ms with an additional 19 MB memory usage.

Future iterations of this work may include expanding the diagnostic dataset and developing a multimodal diagnostic framework that combines both image and textual data to enable more comprehensive and accurate healthcare assessments.

Index Terms—Artificial Intelligence, Mobile Healthcare, TensorFlow Lite, Flutter, Computer Vision, Privacy-Preserving AI

INTRODUCTION

Healthcare services is one of the most critical areas that directly affects society welfare. However, consistently increasing population density, limited healthcare resources and ignorant hospital visits make the sustainability of the health system difficult. As a result of this problem, workload of doctors increase time to diagnosis and decrease patient satisfaction. On the other hand, AI's contribution to our daily life increases demand on modern digital solutions. People expect to assess symptoms and receive feedbacks for their minor health concerns. And AI technology has potential to satisfy this demand and support the healthcare system [1].

Healthcare applications built in the past aim to support healthcare process by storing patients' data in digital environment. However, most existing systems only offer data recording or telemedicine functions; AI-supported diagnosis or treatment guidance features remain limited [2]. This situation

usually only makes the recording process digital instead of reducing the burden on healthcare professionals. Also, lack of privacy and security of user data is another important factor limiting the common use of such applications.

Research in different fields shows that proper integration of AI will support professional processes. For example, LLMs such as "LegalGuardian" [3] shows that securely integrated AI solutions can reduce professional workload while offering reliable services in legal processes. A similar approach can support healthcare services to reduce the workload on doctors and improve service quality.

The gap in practical can be filled with a AI-supported healthcare mobile application that provides user privacy. Existing systems usually handle diagnosis on external servers, which makes them unsuitable for resource-limited settings and raises data privacy concerns.

dAIgnose, introduced by this study, is a mobile-based healthcare application developed with Flutter and TensorFlow Lite. This application allows users to perform initial symptom assessments using on device operating AI analysis even without requiring internet connection. dAIgnose aims to reduce ignorant hospital visits and assist health professionals in managing patient workload by combining accessibility, privacy, and diagnostic support.

RELATED WORK

Many studies have explored AI supported mobile applications in the healthcare field.

Some of these studies focus on providing symptom checking through text-based user input. However, image based diagnosis isn't supported and it was desperately rely on user input, which can reduce diagnostic accuracy.

Other research aims to enable remote doctor appointments. Solutions designed to make doctor-patient communication easier and remotely, known as telemedicine, require professional involvement in all cases and do not allow self-diagnosis [4].

Some other studies have aimed to achieve high-accuracy results in image-based diagnosis processes using deep learning models. However, these studies have generally remain limited to laboratory environment and have not been integrated into user-friendly mobile platforms.

For example, Wu et al. (2025) [5] developed a deep learning model to detect diseases from medical images. The model was trained to classify lung diseases (e.g., pneumonia and

tuberculosis). The study achieved high accuracy rates in a laboratory environment. However, the system was only tested on controlled data and has not been integrated into a mobile or user-accessible platform. For this reason, the model's performance has not been evaluated in real-world settings.

In this context, dAIgnose, developed using TensorFlow Lite, provides AI-based image diagnosis running on the device, resulting in less dependence on the internet connection and offering privacy protection. Additionally, features such as appointment scheduling and location services allow users to access professional healthcare services directly.

METHODOLOGY

In this section, system architecture, workflow and design justification of dAIgnose are explained. Firstly, system overview will be introduced, then layered structure and workflow will be explained. Finally, technologies preferred will be reasoned.

dAIgnose is a mobile application that enables users to upload images of their health-related symptoms and receive AI-powered diagnoses. The system aims to reduce the workload on healthcare professionals and prevent unnecessary hospital visits. Flutter-based application can run on multiple platforms. Google Firebase services ensures authentication and database operations executed securely. On-device diagnose model with TensorFlow Lite provides extra privacy and faster diagnosis.

This application consists of three layer: (1) presentation layer, (2) service layer and (3) infrastructure layer.

- **Presentation Layer:** The Presentation Layer is responsible for the interaction between the user and the system. Built using Flutter, it provides a consistent user experience across Android and iOS platforms. This layer includes interfaces for image loading, viewing diagnostic results, scheduling appointments, and navigating to medical facilities. It is designed with usability, responsiveness, and accessibility. The interface adapts to different screen sizes and devices. Validation mechanisms and real-time feedback help minimize input errors and increase user experience.
- **Service Layer:** The Service Layer handles data management between Presentation and Infrastructure layers. It also aims to reduce dependencies in the application architecture and increase code reusability by abstracting the communication between the UI and the database or authentication services. In this layer, separate service classes are defined for each functional area. For example, AuthService class manages authentication processes by communicating Firebase, while the HospitalService class manages hospital-related functionalities only. This modular approach allows a clear separation of concerns and increase maintainability.

- **Infrastructure Layer:** The infrastructure layer is responsible for the application's interaction with external services and diagnosis model. In this layer, authentication and data storage operations are managed through Google services such as Firebase Authentication and Cloud Firestore. Other application layers can execute operations without understanding the technical details of these services.

The diagnostic model was implemented using a sequential CNN architecture that includes multiple convolution and pooling layers. Each convolutional block uses ReLU activation and dropout to prevent overfitting. The network ends with fully connected dense layers and a sigmoid output layer for classification. The model was trained with TensorFlow and later converted to TensorFlow Lite for on-device inference.

This TensorFlow Lite model evaluates image uploaded by user to classify diagnosis. This operation runs on-device and doesn't require any other API call. In this way, data privacy enhances and evaluation time significantly decreases.

With this structure, the application's infrastructure dependencies are collected in a single layer, ensuring modularity, scalability, and maintainability of the system.

Since Firebase Authentication and Firestore are integrated into Infrastructure layer, there is no need for a separate Data Layer in this architecture.

System Workflow

The general workflow of the application covers the process from the user logging into the system to getting the diagnosis result. The process relies on data flow between multi-layered architecture.

Firstly, user verifies their identity using their username and password. This operation handled by Firebase Authentication service declared in Infrastructure Layer. After successful login, main menu UI displays based on user's role as a result of state management.

In main menu, user logged in as patient clicks disease diagnosis button and the selects specific diagnosis type (skin, oral and nail). User uploads image to be diagnosed. Then, pre-trained related model loads in and image provided gets preprocessed for diagnose. On-device operating TensorFlow Lite analysis given image and generates a diagnosis result based on image's classification. Since the process is performed on the device, data privacy is protected, network delays are minimized, and the system can respond more quickly.

Obtained result is shown to user both textual and visual form. Additionally, confidence rate of diagnosis is also displayed. This way, the diagnosis process is completes.

Design Justification

Used technologies in this study are selected to cover the system's weaknesses. Main goal of this study is to provide users fast, secure and accessible diagnosis service while reducing workload of healthcare professionals. In this way, every decision made during development is based on certain reasons.

Flutter technology was chosen because it provides multi-platform support, is efficient in performance, and simplifies UI development.

TensorFlow Lite was chosen because it enables fast, on-device analysis even on mobile devices with low processing capacity. This approach both protects data privacy and reduces dependence on internet connection.

Google Firebase services is used because provides reliable and scalable authentication and database operations.

Together , the application has been developed with a layered structure that prioritizes scalability, security, and maintainability.

The NoSQL data structure used in the application, representing main entities and their document ID–based relationships. Each collection (e.g., hospital, patient, appointment) is independent, and references between documents are maintained through ID fields rather than nested subcollections.

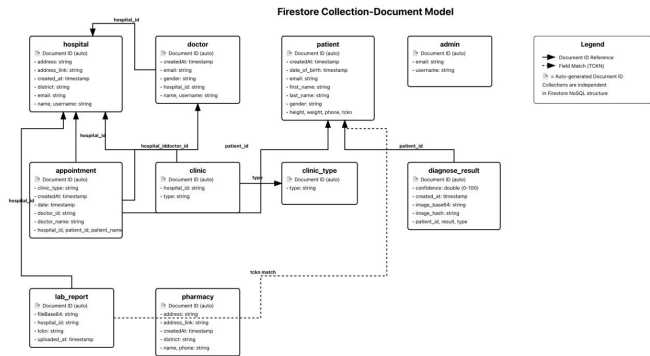


Fig. 1. Firestore Collection–Document Model

EXPERIMENTS AND RESULTS

In this section, the performance of the dAignose has been evaluated on an actual device. In the experiments, the model’s classification accuracy, memory, and processing performance were evaluated.

Experimental Setup

Application was tested on iPhone 15 Pro (iOS 18.5).

Two different test datasets were used: validation data and real-world data. Validation data was collected from the training set’s webpage on Kaggle. Real-world data was obtained from different internet sources.

Accuracy Evaluation

According to the test results, the accuracy rate of the validation data was 71%, while the accuracy rate of the real-world data was 30%.

This difference shows that the model has low generalization performance under real-world conditions.

The average confidence rate is around 90% for correctly classified examples in both datasets.

Dataset	Skin (%)	Oral (%)	Nail (%)
Validation/Test	60	71	83
Real-world	20	44	28

System Performance

Approximately 19 MB of additional RAM usage was observed during the process from model loading to completion of the diagnosis. The processing time was measured at an average of 10 ms, while CPU usage was measured at 186%.

These values indicate that the model can run at high speed on mobile devices but may create a heavy processor load on devices with lower specifications.

Discussion

The results show that the model can operate with high accuracy on controlled data but that its performance drops significantly under real-world conditions.

This indicates that the training dataset needs to be diversified and the model retrained with more real-world examples.

In the future, it is planned to work on multimodal approaches that combine visual and textual data in order to improve performance and improve the diagnostic process.

CONCLUSION

This study introduces dAignose mobile application that designed to reduce the workload in the healthcare system and enable patients to pre-assess their own conditions. The application provides diagnostic support for skin, nail, and oral diseases using a TensorFlow Lite-based image classification model.

Real-device testing revealed that the model’s accuracy rate is 71% on validation data and reached 20% on real-world data.

This result shows that the model is successful under controlled conditions but has low generalization capacity in real-world examples.

However, the system’s performance with only 19 MB of additional memory usage and a processing time of 10 ms shows that it works fast and efficiently on mobile devices.

In the future, it is planned to expand the training data set to improve the system’s accuracy and provide visual-textual (multimodal) analysis support.

This aims to enable users to make a more accurate pre-diagnosis based on both visual and text.

REFERENCES

- [1] L. Aissaoui Ferhi, M. Ben Amar, F. Choubani, and R. Bouallegue, "Enhancing diagnostic accuracy in symptom-based health checkers: a comprehensive machine learning approach with clinical vignettes and benchmarking," *Front. Artif. Intell.*, vol. 7, Art. no. 1397388, Oct. 2024, doi: 10.3389/frai.2024.1397388.
- [2] Y. A. Fahim, I. W. Hasani, S. Kabba, and W. M. Ragab, "Artificial intelligence in healthcare and medicine: clinical applications, therapeutic advances, and future perspectives," *Eur. J. Med. Res.*, vol. 30, no. 1, Art. no. 848, Sep. 2025, doi: 10.1186/s40001-025-03196-w.
- [3] M. M. Demir, H. T. Otal, and M. A. Canbaz, "LegalGuardian: A privacy-preserving framework for secure integration of large language models in legal practice," *arXiv:2501.10915*, Jan. 2025.

- [4] A. Singh and L. Zhao, "Telemedicine systems and the challenge of self-diagnosis," *J. Med. Syst.*, vol. 49, no. 2, pp. 112–120, Feb. 2025.
- [5] Y. Wu, Z. Li, and H. Zhang, "Deep learning-based image recognition technology in medical diagnosis," *Appl. Math. Nonlinear Sci.*, vol. 10, no. 1, Art. no. 1008, 2025.